MTM4692

Spring 2026

Fettah KIRAN
Department of AI and Data Engineering
fkiran@yildiz.edu.tr

Views

Virtual Tables, Security Abstraction & Query Simplification

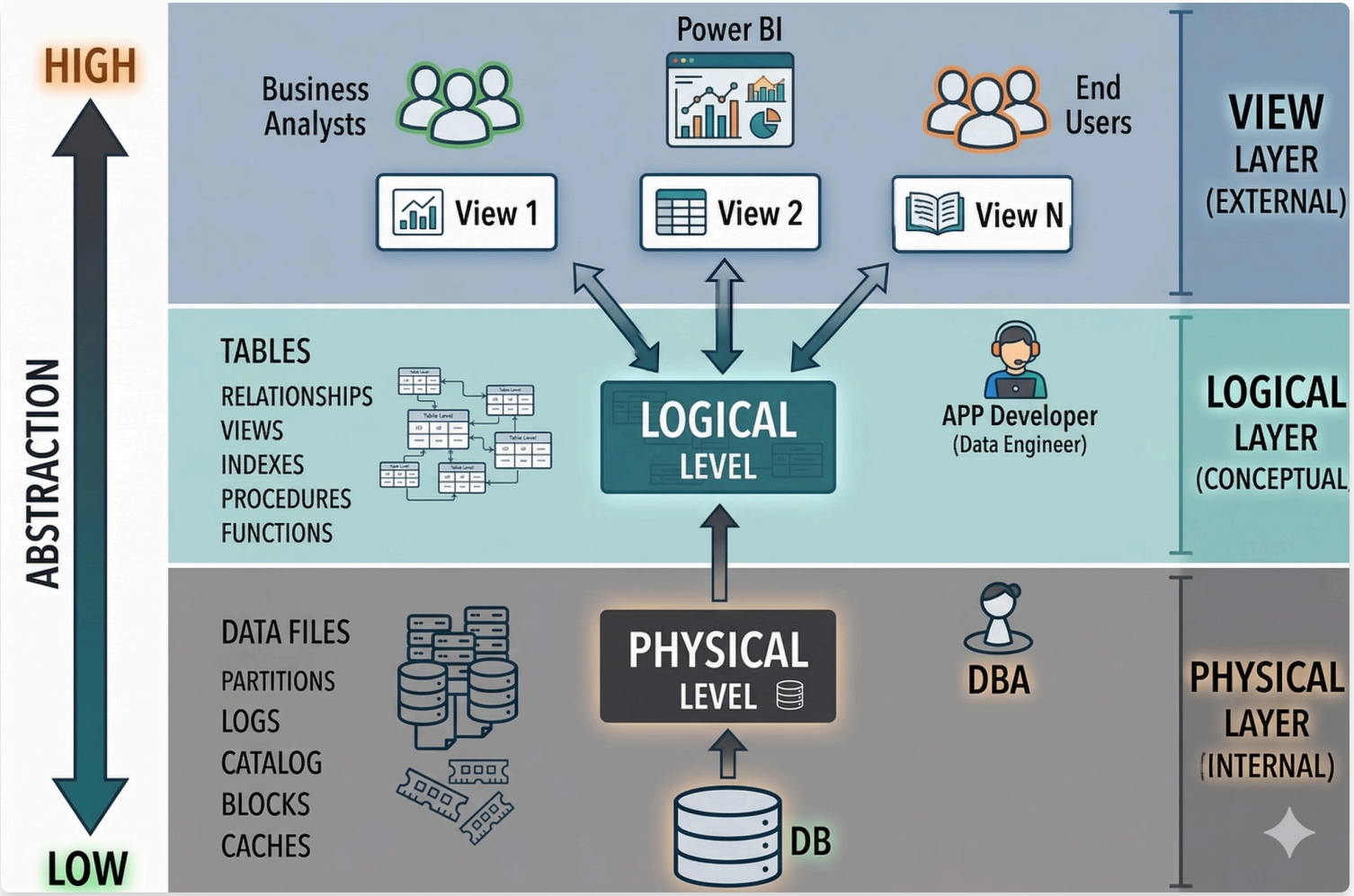
📋 Today's Agenda (1:00 PM – 3:45 PM)

Time	Topic	Type
1:00 – 1:15	Week 4 recap poll	Review
1:15 – 1:50	What are Views? Types & Benefits	**Theory** 📖
1:50 – 2:15	Creating & Managing Views	**Theory** 📖
2:15 – 2:30	☕ Break (15 min)	
2:30 – 2:55	Updatable Views & Restrictions	**Theory** 📖
2:55 – 3:10	View Security & Best Practices	**Theory** 📖
3:10 – 3:40	Hands-on: Views for University & E-Commerce	**Practice** 💻
3:40 – 3:45	Recap & next week preview	Wrap-up

Week 4 Quick Recap

Concept	Key Point
B-Tree Index	Balanced tree → O(log n) lookup instead of O(n) scan
EXPLAIN QUERY PLAN	Shows SCAN (slow) vs SEARCH (fast)
Composite Index	Leftmost prefix rule — order of columns matters
Covering Index	All needed columns in index → no table lookup
Partial Index	Indexes only rows matching a WHERE condition
Expression Index	Index on computed values: `LOWER(email)`
Anti-patterns	Functions on indexed columns, leading `%` wildcards
ANALYZE	Collects stats for the query optimizer

DB 3-Level Architecture



https://www.blog.datawithbaraa.com/p/sql-views-the-hidden-gem-of-database

📖 PART 1 — THEORY

What Are Views?

Views: Virtual Tables

A **view** is a named SELECT query stored in the database.

```sql
CREATE VIEW honor_students AS
SELECT student_id, first_name, last_name, gpa
FROM student
WHERE gpa >= 3.5;
```

Using the view (looks like a table):

```sql
SELECT * FROM honor_students;
SELECT COUNT(*) FROM honor_students;
SELECT * FROM honor_students WHERE last_name LIKE 'Y%';
```

> 💡 A view **does not store data**. It's like a saved query that runs each time you access it.

Views vs Tables

Feature	Table	View
Stores data?	✅ Yes (on disk)	❌ No (query re-runs each time)
Occupies space?	✅ Yes	❌ Minimal (just the query definition)
Can INSERT/UPDATE?	✅ Always	⚠️ Sometimes (simple views only)
Has indexes?	✅ Yes	❌ No (uses underlying table indexes)
Schema visible?	✅ All columns	✅ Only selected columns
Performance?	Direct disk read	Re-executes query each time

> 📝 Views are sometimes called **virtual tables**.

Why Use Views?

1. 🔒 Data Security — Column/Row Level Access Control

```sql
-- HR can see all data
SELECT * FROM employee;

-- Regular users only see name and department (no salary!)
CREATE VIEW public_employees AS
SELECT emp_id, first_name, last_name, department
FROM employee;
-- GRANT SELECT ON public_employees TO general_user;
```

2. 📦 Simplification — Hide Complex Queries

```sql
-- Complex JOIN used frequently
CREATE VIEW student_transcript AS
SELECT s.first_name, s.last_name, c.course_name, e.grade
FROM student s
JOIN enrollment e ON s.student_id = e.student_id
JOIN course c ON e.course_id = c.course_id;

-- Simple usage
SELECT * FROM student_transcript WHERE last_name = 'Kaya';
```

Why Use Views? (continued)

3. 🔄 Consistency — One Definition, Many Users

```sql
-- Definition changes in one place, all users see the update
CREATE VIEW active_orders AS
SELECT * FROM orders WHERE status IN ('pending', 'processing');
```

Why Use Views? (continued)

4. 📊 Reporting — Pre-built Report Queries

```sql
CREATE VIEW sales_summary AS
SELECT
    strftime('%Y-%m', order_date) AS month,
    COUNT(*) AS order_count,
    SUM(total_amount) AS revenue
FROM orders
GROUP BY strftime('%Y-%m', order_date);

-- Simple report access
SELECT * FROM sales_summary ORDER BY month;
```

🧠 Exam Focus: Views

Q: Which of the following is TRUE about views?

A) Views store data in the database like tables

B) **Views execute their underlying query each time they are accessed** ✅

C) Views cannot be used in JOIN operations

D) Views are faster than tables because they cache data

Q: Which is a valid benefit of using views?

A) Views automatically improve query performance

B) Views can replace all tables in a database

C) **Views can restrict user access to specific columns of a table** ✅

D) Views always store computed results for faster access

Creating & Managing Views

CREATE VIEW Syntax

```sql
-- Basic syntax
CREATE VIEW view_name AS
SELECT column1, column2, ...
FROM table_name
WHERE condition;

-- With column aliases
CREATE VIEW view_name (alias1, alias2, ...) AS
SELECT column1, column2, ...
FROM table_name;

-- IF NOT EXISTS
CREATE VIEW IF NOT EXISTS view_name AS
SELECT ...;
```

View Examples

```sql
-- 1. Simple filtering view
CREATE VIEW cs_students AS
SELECT s.student_id, s.first_name, s.last_name, s.gpa
FROM student s
JOIN department d ON s.dept_id = d.dept_id
WHERE d.dept_name = 'Computer Science';

-- 2. Aggregation view
CREATE VIEW dept_statistics AS
SELECT d.dept_name,
       COUNT(s.student_id) AS student_count,
       ROUND(AVG(s.gpa), 2) AS avg_gpa,
       MAX(s.gpa) AS max_gpa,
       MIN(s.gpa) AS min_gpa
FROM department d
LEFT JOIN student s ON d.dept_id = s.dept_id
GROUP BY d.dept_id, d.dept_name;

-- 3. Multi-table view
CREATE VIEW full_enrollment AS
SELECT s.first_name || ' ' || s.last_name AS student,
       d.dept_name, c.course_name, c.credits,
       i.first_name || ' ' || i.last_name AS instructor,
       e.grade, e.semester
FROM enrollment e
JOIN student s ON e.student_id = s.student_id
JOIN course c ON e.course_id = c.course_id
JOIN department d ON s.dept_id = d.dept_id
LEFT JOIN instructor i ON c.instructor_id = i.instructor_id;
```

Managing Views

```sql
-- List all views
SELECT name FROM sqlite_master WHERE type = 'view';

-- See view definition
SELECT sql FROM sqlite_master
WHERE type = 'view' AND name = 'honor_students';

-- Or use dot command
.schema honor_students

-- Drop a view
DROP VIEW IF EXISTS honor_students;

-- Recreate (SQLite has no ALTER VIEW - drop and recreate)
DROP VIEW IF EXISTS honor_students;
CREATE VIEW honor_students AS
SELECT student_id, first_name, last_name, gpa
FROM student
WHERE gpa >= 3.7;   -- Changed from 3.5 to 3.7
```

Views on Views (Nested Views)

```sql
-- Base view: all enrollments
CREATE VIEW v_enrollments AS
SELECT s.student_id, s.first_name, s.last_name,
       c.course_name, e.grade
FROM student s
JOIN enrollment e ON s.student_id = e.student_id
JOIN course c ON e.course_id = c.course_id;

-- View on view: only A grades
CREATE VIEW v_top_performers AS
SELECT * FROM v_enrollments
WHERE grade = 'A';

-- Use the nested view
SELECT * FROM v_top_performers;
```

> ⚠️ Be careful with deeply nested views — they can be hard to debug and may have performance implications.

🧠 Exam Focus: CREATE VIEW

> **Q:** How do you change the definition of an existing view in SQLite?
>
> A) `ALTER VIEW view_name AS SELECT ...`
>
> B) `MODIFY VIEW view_name AS SELECT ...`
>
> C) **Drop the view and recreate it with the new definition** ✅
>
> D) `UPDATE sqlite_master SET sql = ... WHERE name = 'view_name'`

💡 SQLite does not support `ALTER VIEW`. You must `DROP VIEW` and then `CREATE VIEW` again.

Updatable Views

Can You INSERT/UPDATE/DELETE Through a View?

In SQLite:

> **Yes**, but ONLY for **simple views** that map directly to a single table.

Rules for updatable views in SQLite:

1. The view must reference exactly **one table**
2. No **aggregate functions** (COUNT, SUM, AVG, etc.)
3. No **DISTINCT** clause
4. No **GROUP BY** or **HAVING**
5. No **subqueries** in the SELECT list
6. No **set operations** (UNION, INTERSECT, EXCEPT)

Updatable View Example (Theory)

> 💡 **SQLite Note:** While simple views *technically* qualify for updates based on the rules, **most UPDATE/INSERT/DELETE operations require INSTEAD OF triggers** in practice.

```sql
-- Simple view on one table (meets all updatable view criteria)
CREATE VIEW active_students AS
SELECT student_id, first_name, last_name, email, gpa
FROM student
WHERE gpa >= 2.0;

-- ⚠️ In theory, these should work on simple views:
-- INSERT INTO active_students (first_name, last_name, email, gpa)
-- VALUES ('New', 'Student', 'new@ytu.edu.tr', 3.5);

-- ⚠️ But in SQLite, direct updates may fail.
-- Solution: Use INSTEAD OF triggers (see next slide)
```

Non-Updatable Views

```sql
-- ❌ Cannot INSERT/UPDATE - uses JOIN (multiple tables)
CREATE VIEW student_courses AS
SELECT s.first_name, c.course_name, e.grade
FROM student s
JOIN enrollment e ON s.student_id = e.student_id
JOIN course c ON e.course_id = c.course_id;

-- ❌ Cannot INSERT/UPDATE - uses aggregation
CREATE VIEW dept_stats AS
SELECT dept_id, COUNT(*) as cnt, AVG(gpa) as avg_gpa
FROM student
GROUP BY dept_id;

-- ❌ Cannot INSERT/UPDATE - uses DISTINCT
CREATE VIEW unique_departments AS
SELECT DISTINCT dept_id FROM student;
```

WITH CHECK OPTION (MySQL Concept)

> 💡 SQLite does NOT support `WITH CHECK OPTION`, but it's an important exam concept.

```sql
-- In MySQL --
CREATE VIEW honor_students AS
SELECT student_id, first_name, last_name, gpa
FROM student WHERE gpa >= 3.5
WITH CHECK OPTION;

-- This INSERT would FAIL because gpa < 3.5
INSERT INTO honor_students (first_name, last_name, gpa) VALUES ('Weak', 'Student', 2.0);
-- ERROR: CHECK OPTION failed
```

> Without `WITH CHECK OPTION`

- The INSERT would succeed, but the new row would **disappear** from the view
- The row exists in the table but doesn't match `WHERE gpa >= 3.5`

🧠 Exam Focus: WITH CHECK OPTION

> **Q1:** What does `WITH CHECK OPTION` do for a view?
>
> A) Improves query performance
>
> B) **Prevents INSERT/UPDATE operations that would create rows not visible through the view** ✅
>
> C) Adds a CHECK constraint to the underlying table
>
> D) Enables the view to be updated

> **Q2:** What happens if you INSERT a row through a view (without WITH CHECK OPTION) that doesn't match the view's WHERE clause?
>
> A) The INSERT fails with an error
> B) **The row is inserted into the table but not visible through the view** ✅
> C) The row is inserted and visible through the view
> D) The view is automatically updated to include the row

INSTEAD OF Triggers (Required for Updatable Views)

```sql
-- Simple view on one table
CREATE VIEW student_simple AS
SELECT student_id, first_name, last_name
FROM student
WHERE dept_id IS NOT NULL;

-- ✅ INSTEAD OF INSERT trigger
CREATE TRIGGER tr_insert_student_simple
INSTEAD OF INSERT ON student_simple
BEGIN
    INSERT INTO student (student_id, first_name, last_name, email, dept_id)
    VALUES (NEW.student_id, NEW.first_name, NEW.last_name,
            NEW.first_name || '.' || NEW.last_name || '@ytu.edu.tr', 1);
END;

-- ✅ INSTEAD OF UPDATE trigger
CREATE TRIGGER tr_update_student_simple
INSTEAD OF UPDATE ON student_simple
BEGIN
    UPDATE student
    SET first_name = NEW.first_name, last_name = NEW.last_name
    WHERE student_id = OLD.student_id;
END;

-- ✅ INSTEAD OF DELETE trigger
CREATE TRIGGER tr_delete_student_simple
INSTEAD OF DELETE ON student_simple
BEGIN
    DELETE FROM student WHERE student_id = OLD.student_id;
END;

-- Now these work:
INSERT INTO student_simple (student_id, first_name, last_name)
VALUES (100, 'John', 'Doe');

UPDATE student_simple SET last_name = 'Smith' WHERE student_id = 100;
DELETE FROM student_simple WHERE student_id = 100;
```

Key Point: Even simple views that meet all updatable criteria still require INSTEAD OF

Complex Views with INSTEAD OF Triggers

JOINs, aggregations, etc.):

```sql
-- View with JOIN (not updatable by default)
CREATE VIEW student_full AS
SELECT s.student_id, s.first_name, s.last_name,
       d.dept_name
FROM student s
JOIN department d ON s.dept_id = d.dept_id;

-- ✅ Use trigger to make it updatable
CREATE TRIGGER tr_update_student_full
INSTEAD OF UPDATE ON student_full
BEGIN
    UPDATE student
    SET first_name = NEW.first_name,
        last_name = NEW.last_name
    WHERE student_id = OLD.student_id;
END;

-- Now this works despite the JOIN:
UPDATE student_full SET first_name = 'Alice'
WHERE student_id = 1;
```

☕ BREAK (10 minutes)

View Security & Best Practices

Views for Security: Column-Level

```sql
-- Full employee table (sensitive data)
CREATE TABLE employee (
    emp_id INTEGER PRIMARY KEY,
    first_name TEXT, last_name TEXT,
    email TEXT, salary REAL,
    ssn TEXT, department TEXT
);

-- Public view: hides salary and SSN
CREATE VIEW v_employee_public AS
SELECT emp_id, first_name, last_name, email, department
FROM employee;

-- HR view: includes salary but not SSN
CREATE VIEW v_employee_hr AS
SELECT emp_id, first_name, last_name, email,
       salary, department
FROM employee;

-- In production, you would:
-- GRANT SELECT ON v_employee_public TO general_user;
-- GRANT SELECT ON v_employee_hr TO hr_role;
-- REVOKE SELECT ON employee FROM general_user;
```

Views for Security: Row-Level

```sql
-- Only show students from a specific department
CREATE VIEW v_cs_students AS
SELECT * FROM student
WHERE dept_id = (SELECT dept_id FROM department
                 WHERE dept_name = 'Computer Science');

-- Only show recent orders
CREATE VIEW v_recent_orders AS
SELECT * FROM orders
WHERE order_date >= date('now', '-30 days');

-- Only show non-cancelled orders
CREATE VIEW v_active_orders AS
SELECT * FROM orders
WHERE status != 'cancelled';
```

> 💡 Row-level views are like having a permanent WHERE filter.

View Performance Considerations

Views do NOT store data — the query runs every time:

```sql
-- This view runs a complex JOIN + aggregation each time
CREATE VIEW expensive_report AS
SELECT d.dept_name, COUNT(*) AS cnt,
       AVG(s.gpa) AS avg_gpa
FROM student s
JOIN department d ON s.dept_id = d.dept_id
GROUP BY d.dept_id;

-- Every access re-executes the full query
SELECT * FROM expensive_report;  -- runs the JOIN + GROUP BY
SELECT * FROM expensive_report;  -- runs it AGAIN
```

Performance tips:

- Use indexes on the underlying tables
- Avoid deeply nested views (view on view on view)
- For read-heavy reports, consider **materialized views** (not native in SQLite)

Materialized Views (Concept)

> SQLite does not support materialized views natively, but you can simulate them.

Concept: Store the view result as a table

```sql
-- Create a "materialized" view manually
CREATE TABLE mv_dept_stats AS SELECT d.dept_name, COUNT(*) AS cnt, AVG(s.gpa) AS avg_gpa
FROM student s JOIN department d ON s.dept_id = d.dept_id GROUP BY d.dept_id;

-- Fast reads (it's a real table)
SELECT * FROM mv_dept_stats;

-- Must refresh manually when data changes
DROP TABLE mv_dept_stats;
CREATE TABLE mv_dept_stats AS ...;
```

Materialized Views (Concept)

Feature	Regular View	Materialized View
Data storage	No	Yes
Always up-to-date	Yes	No (needs refresh)
Read performance	Slower	Faster
Write overhead	None	Refresh needed

🧠 Exam Focus: Materialized Views

> **Q:** What is the main difference between a regular view and a materialized view?
>
> A) A regular view is faster than a materialized view
>
> B) **A materialized view stores the query result while a regular view re-executes the query each time** ✅
>
> C) Regular views support WITH CHECK OPTION but materialized views don't
>
> D) Materialized views are always up-to-date

View Best Practices

✅ **Do:**

1. **Name views clearly**: `v_` or `vw_` prefix (e.g., `v_active_orders`)
2. **Use views for security**: Hide sensitive columns
3. **Use views for simplification**: Wrap complex JOINs
4. **Document the purpose**: Add comments explaining why the view exists
5. **Keep views shallow**: Avoid nesting more than 2 levels

❌ Don't:

1. **Don't use views as a substitute for indexes** (views don't help performance)
2. **Don't nest views deeply** (hard to debug and optimize)
3. **Don't assume views are updatable** without checking
4. **Don't create views for trivial queries** (overhead without benefit)
5. **Don't forget to update views** when underlying tables change

PART 4 — CODING PRACTICE

Views for the University Database

Practice: Basic Views

```sql
-- 1. Student directory view (hides internal IDs)
CREATE VIEW v_student_directory AS
SELECT first_name, last_name, email
FROM student
ORDER BY last_name, first_name;

SELECT * FROM v_student_directory;

-- 2. Honor roll view
CREATE VIEW v_honor_roll AS
SELECT s.first_name, s.last_name, s.gpa,
       d.dept_name
FROM student s
JOIN department d ON s.dept_id = d.dept_id
WHERE s.gpa >= 3.5
ORDER BY s.gpa DESC;

SELECT * FROM v_honor_roll;
```

💻 Practice: Complex Views

```sql
-- 3. Full transcript view
CREATE VIEW v_transcript AS
SELECT s.student_id,
       s.first_name || ' ' || s.last_name AS student_name,
       d.dept_name,
       c.course_code, c.course_name, c.credits,
       e.grade, e.semester,
       i.first_name || ' ' || i.last_name AS instructor_name
FROM enrollment e
JOIN student s ON e.student_id = s.student_id
JOIN course c ON e.course_id = c.course_id
JOIN department d ON s.dept_id = d.dept_id
LEFT JOIN instructor i ON c.instructor_id = i.instructor_id;

-- Use it simply
SELECT * FROM v_transcript WHERE student_id = 1;
SELECT * FROM v_transcript WHERE dept_name = 'Computer Science';
SELECT * FROM v_transcript WHERE grade = 'A';
```

Practice: Aggregation Views

```sql
-- 4. Department statistics
CREATE VIEW v_dept_stats AS
SELECT d.dept_name,
       COUNT(DISTINCT s.student_id) AS student_count,
       ROUND(AVG(s.gpa), 2) AS avg_gpa,
       COUNT(DISTINCT c.course_id) AS course_count,
       COUNT(DISTINCT i.instructor_id) AS instructor_count
FROM department d
LEFT JOIN student s ON d.dept_id = s.dept_id
LEFT JOIN course c ON d.dept_id = c.dept_id
LEFT JOIN instructor i ON d.dept_id = i.dept_id
GROUP BY d.dept_id, d.dept_name;

SELECT * FROM v_dept_stats;

-- 5. Course enrollment summary
CREATE VIEW v_course_enrollment AS
SELECT c.course_code, c.course_name,
       COUNT(e.student_id) AS enrolled_count,
       GROUP_CONCAT(e.grade) AS grades
FROM course c
LEFT JOIN enrollment e ON c.course_id = e.course_id
GROUP BY c.course_id, c.course_code, c.course_name;

SELECT * FROM v_course_enrollment;
```

💻 Practice: Security Views

```sql
-- 6. Public student view (no GPA, no personal details)
CREATE VIEW v_student_public AS
SELECT student_id, first_name, last_name
FROM student;

-- 7. Advisor view (includes GPA for advising)
CREATE VIEW v_student_advisor AS
SELECT s.student_id, s.first_name, s.last_name,
       s.gpa, d.dept_name
FROM student s
JOIN department d ON s.dept_id = d.dept_id;

-- 8. Registrar view (full enrollment details)
CREATE VIEW v_registrar AS
SELECT s.student_id, s.first_name, s.last_name,
       c.course_code, c.course_name,
       e.grade, e.semester
FROM enrollment e
JOIN student s ON e.student_id = s.student_id
JOIN course c ON e.course_id = c.course_id;
```

Practice: E-Commerce Views

```sql
-- Open ecommerce.db
-- 9. Product catalog view
CREATE VIEW v_product_catalog AS
SELECT p.product_id, p.product_name,
       p.price, p.stock_quantity,
       c.category_name
FROM product p
JOIN category c ON p.category_id = c.category_id
WHERE p.stock_quantity > 0;

-- 10. Customer order history
CREATE VIEW v_order_history AS
SELECT c.first_name || ' ' || c.last_name AS customer,
       o.order_id, o.order_date, o.status,
       o.total_amount
FROM orders o
JOIN customer c ON o.customer_id = c.customer_id;

-- 11. Revenue report
CREATE VIEW v_monthly_revenue AS
SELECT strftime('%Y-%m', order_date) AS month,
       COUNT(*) AS order_count,
       ROUND(SUM(total_amount), 2) AS revenue
FROM orders
GROUP BY strftime('%Y-%m', order_date);
```

💻 Practice: Using Views in Queries

```sql
-- Views can be used in JOINs
SELECT t.student_name, t.course_name, t.grade
FROM v_transcript t
JOIN v_honor_roll h ON t.student_name = h.first_name || ' ' || h.last_name;

-- Views can be used in subqueries
SELECT * FROM v_dept_stats
WHERE avg_gpa > (SELECT AVG(avg_gpa) FROM v_dept_stats);

-- Views can be filtered and sorted
SELECT * FROM v_course_enrollment
WHERE enrolled_count > 5
ORDER BY enrolled_count DESC;

-- Checking view definitions
SELECT name, sql FROM sqlite_master WHERE type = 'view';
```

💻 Practice: Updatable View Test

```sql
-- Simple single-table view → updatable
CREATE VIEW v_students_simple AS
SELECT student_id, first_name, last_name, gpa
FROM student;

-- ✅ This INSERT works
INSERT INTO v_students_simple (first_name, last_name, gpa)
VALUES ('Test', 'Student', 3.0);

-- ✅ This UPDATE works
UPDATE v_students_simple SET gpa = 3.2
WHERE first_name = 'Test' AND last_name = 'Student';

-- ✅ This DELETE works
DELETE FROM v_students_simple
WHERE first_name = 'Test' AND last_name = 'Student';

-- ❌ Multi-table view → NOT updatable
-- INSERT INTO v_transcript (student_name, course_name)
-- VALUES ('X', 'Y');   -- ERROR!
```

Practice: INSTEAD OF Trigger

```sql
-- Make a multi-table view insertable
CREATE VIEW v_student_dept AS
SELECT s.student_id, s.first_name, s.last_name,
       d.dept_name
FROM student s
JOIN department d ON s.dept_id = d.dept_id;

-- INSTEAD OF INSERT trigger
CREATE TRIGGER tr_insert_student_dept
INSTEAD OF INSERT ON v_student_dept
BEGIN
    INSERT INTO student (first_name, last_name, dept_id)
    VALUES (
        NEW.first_name, NEW.last_name,
        (SELECT dept_id FROM department
         WHERE dept_name = NEW.dept_name)
    );
END;

-- Now this works through the trigger:
INSERT INTO v_student_dept (first_name, last_name, dept_name)
VALUES ('Trigger', 'Test', 'Computer Science');

-- Verify
SELECT * FROM v_student_dept WHERE first_name = 'Trigger';
```

💻 Practice: "Materialized" View in SQLite

```sql
-- Simulate a materialized view
CREATE TABLE mv_student_summary AS
SELECT d.dept_name,
       COUNT(*) AS student_count,
       ROUND(AVG(s.gpa), 2) AS avg_gpa,
       MAX(s.gpa) AS max_gpa
FROM student s
JOIN department d ON s.dept_id = d.dept_id
GROUP BY d.dept_id, d.dept_name;

-- Fast read
SELECT * FROM mv_student_summary;

-- Refresh function (drop and recreate)
DROP TABLE IF EXISTS mv_student_summary;
CREATE TABLE mv_student_summary AS
SELECT d.dept_name,
       COUNT(*) AS student_count,
       ROUND(AVG(s.gpa), 2) AS avg_gpa,
       MAX(s.gpa) AS max_gpa
FROM student s
JOIN department d ON s.dept_id = d.dept_id
GROUP BY d.dept_id, d.dept_name;
```

Summary: Views Cheat Sheet

Operation	SQL
Create	`CREATE VIEW v AS SELECT ...`
Use	`SELECT * FROM v WHERE ...`
List all	`SELECT name FROM sqlite_master WHERE type='view'`
See definition	`SELECT sql FROM sqlite_master WHERE name='v'`
Drop	`DROP VIEW IF EXISTS v`
Modify	Drop + Recreate
Updatable?	Single table, no aggregation/DISTINCT/GROUP BY

> Needs to be watched since next week's quiz will be from this video.
> https://www.youtube.com/watch?v=n17RCiV5xpA

📝 Practice Exercises (Homework)

1. **Create views for the university database:**
   - A view showing each instructor's teaching load (courses and student count)
   - A view showing students who haven't enrolled in any course (LEFT JOIN + IS NULL)
   - A "dashboard" view combining student, course, and department stats
2. **Create views for the e-commerce database:**
   - A product performance view (total sold, total revenue per product)
   - A customer segmentation view (total spent, order count per customer)
   - Test which views are updatable and which are not

🔮 Next Week Preview

Week 6: Relational vs NoSQL Databases

- ACID vs BASE consistency models
- Types of NoSQL databases (Document, Key-Value, Graph, Column)
- CAP Theorem — Consistency, Availability, Partition tolerance
- When to use SQL vs NoSQL
- MongoDB concepts comparison

Preparation:

- 📖 Read about NoSQL databases (Wikipedia or any intro article)
- 💻 Complete this week's view exercises
- 🤔 Think about: *When would a non-relational database be better?*

Thank You!

Questions?